



Midnight Cram Sheet: The Final 3 Patterns

If you see a problem tomorrow that doesn't fit Heaps, Sliding Window, or Kadane's, it will likely be one of these three. Just memorize the structural templates.

1. Custom Sorting (Greedy Algorithms)

When to use it: Problems involving "overlapping intervals," "meeting rooms," or "activity selection" (e.g., maximizing the number of tasks you can do). **The Trick:** You almost always need to sort the data by the **END time** or the **second value**, not the first.

Python Template (Sorting by a specific index)

```
def max_activities(intervals):
    # intervals looks like: [[start1, end1], [start2, end2]]

    # Sort by the END time (index 1) using a lambda function
    intervals.sort(key=lambda x: x[1])

    count = 0
    last_end_time = -1

    for start, end in intervals:
        # If this activity starts after the last one ended, we can do it!
        if start >= last_end_time:
            count += 1
            last_end_time = end

    return count
```

2. 1D Dynamic Programming (The "Take or Skip" Pattern)

When to use it: Problems asking for the "maximum/minimum" where you cannot pick adjacent elements (e.g., "House Robber" or "Maximum Alternating Sum"). **The Trick:** At every step, you have two choices: Take the current number (and skip the next), or Skip the current number (and take the next).

Python Template (House Robber)

```
def rob(nums):
    if not nums: return 0
    if len(nums) == 1: return nums[0]

    # Keep track of the max money if we robbed the previous house,
    # or the house before the previous.
    prev2 = 0
    prev1 = 0

    for num in nums:
        # Choice: Rob THIS house + prev2, OR skip this house and keep prev1
        current = max(prev2 + num, prev1)

        # Shift variables forward for the next loop
```

```
        prev2 = prev1
        prev1 = current

    return prev1
```

3. The Q3 Scavenger: 2D Grid DFS

When to use it: If the Hard question (Q3) gives you a 2D matrix (a grid of 0s and 1s) and asks you to find "islands," "connected components," or "paths." **The Trick:** Use a recursive function to check Up, Down, Left, and Right, and change visited cells to a different symbol so you don't count them twice.

Python Template (Number of Islands)

```
class Solution:
    def numIslands(self, grid: List[List[str]]) -> int:
        if not grid: return 0

        rows, cols = len(grid), len(grid[0])
        island_count = 0

        def dfs(r, c):
            # 1. Base cases (Out of bounds, or water/already visited)
            if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == '0':
                return

            # 2. Mark as visited by changing '1' to '0'
            grid[r][c] = '0'

            # 3. Explore all 4 directions recursively
            dfs(r + 1, c) # Down
            dfs(r - 1, c) # Up
            dfs(r, c + 1) # Right
            dfs(r, c - 1) # Left

        # Loop through every cell in the grid
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1': # Found land!
                    island_count += 1
                    dfs(r, c) # Sink the entire island so we don't count it aga

        return island_count
```